

Aoun (عون)

Intelligent Cancer Patient Monitoring System

Prepared by:
Mohammad Thaher

Contents

Contents

List of Figures	iii
List of Tables.....	iv
List of Symbols	v
Abstract	vi
Chapter 1: Introduction	7
1.1 Current situation and opportunity	7
1.2 Related work.....	7
1.3 Problem statement (limitation of current systems)	7
1.4 Problem solution.....	8
1.5 Project objectives.....	8
1.6 Technology and tools used	9
1.7 Project Plan (Gantt chart, PERT chart).....	9
1.8	9
Chapter 2: Requirements and Analysis	11
2.1 Software Process Model	11
2.2 System scope with explanation.....	12
2.3 Functional Requirements & Non-Functional Requirements.....	12
Functional Requirements	12
Non-Functional Requirements	13
2.4 Use Case Diagram with Use Cases descriptions.....	14
2.5 Activity Diagram	14
Chapter 3: Design.....	16
3.1 Sequence Diagrams	16
3.2 Class Diagram.....	19
Patient	19
SymptomRecord	19
Other Classes	19
3.3 Database ER Diagram.....	19
Chapter 4: System Implementation	22
4.1 Main System Screenshots	22
Login & Sign Up Screen	22
Patient Home Screen.....	23
Symptom Log Screen.....	24
Dashboard Screen	26
Face Wellness Screen.....	27
Chatbot Screen	28
Emergency Screen.....	30
Doctor Dashboard Screen	31
Admin Dashboard Screen	32
4.2 Important code fragments	34
Random Forest Training Pipeline (ml_main.py)	34
EXIF Rotation Fix (wellness_analyzer.py)	34
Feedback-Weighted RAG Retrieval (aoun_rag.py)	34
Medication Guardrail (chatbot_routes.py v6.1)	35
4.3 List of Non-Implemented requirements and reasons	35
Chapter 5: System Evaluation	36
5.1 System Testing.....	36
Black Box Test Cases	36
5.2 System Evaluation	37
ML Model Evaluation	37

Chapter 6: Conclusion & Future Work	41
6.1 Conclusion	41
6.2 Future work.....	41
References	43
Appendix	44
A. Demo Credentials.....	44
B. Service Start Commands	44
C. PKL Model Files	44
D. Glossary	44

List of Figures

- Figure 1: UML Use Case Diagram — all three actors and their associated use cases.....
- Figure 2: Sequence diagram: symptom logging triggering ML prediction and Firebase alert dispatch.....
- Figure 3: Sequence diagram: facial wellness scan pipeline from selfie to illness score.....
- Figure 4: Sequence diagram: RAG chatbot pipeline from message to LLM response with feedback loop.....
- Figure 5: Entity-Relationship Diagram of the aoun_db PostgreSQL database (8 tables)
- Figure 5: Gantt and PERT chart.....
- Figure 6: Sign up and login screens account creation form and role-based sign-in.....
- Figure 7: Patient home screen - current risk level, quick-action buttons, and recent symptom logs.....
- Figure 8: Symptom log screen with 21 sliders and ML prediction result.....
- Figure 9: Risk trend dashboard (fl_chart) and Lifestyle log — daily habits tracking.....
- Figure 10: Face Wellness Check - visual analysis instructions, tips, and capture options.....
- Figure 11: RAG chatbot — welcome state, user query, AI response with source citations and thumbs feedback.....
- Figure 11: RAG chatbot — welcome state, user query, AI response with source citations and thumbs feedback.....
- Figure 13: Doctor dashboard — patient triage list, chatbot feedback analytics, and symptom trend view.....
- Figure 14: Admin dashboard - patient list, approval, and deactivation controls.....
- Figure 15: Alerts screen and Health Recommendations — general reminders, risk-based tips, and wellness guidance.....

List of Tables

Table 1: Comparison of Aoun with existing oncology monitoring platforms.....	
Table 2: Full technology stack.....	
Table 3: GP2 Agile sprint plan.....	
Table 4: Functional requirements traceability matrix (27/28 implemented)	
Table 5: Verified table list from aoun_db (8 tables)	
Table 6: Non-implemented requirements.....	
Table 7: Black-box test cases — 19/20 passed; TC19 deferred (Firebase key required)	

List of Symbols

Symbols	Meaning
RAG	Retrieval-Augmented Generation
SMOTE	Synthetic Minority Over-sampling Technique
ML	Machine Learning
FCM	Fuzzy C-Means (Clustering algorithm)
RF	Random Forest (Machine learning algorithm)
API	Application Programming Interface
EHR	Electronic Health Record
LLM	Large Language Model

Abstract

Aoun (عون, meaning “support” in Arabic, is an intelligent cancer patient monitoring system developed at Applied Science Private University designed to address a critical gap in oncology care: the lack of continuous, real-time patient monitoring between hospital visits. The platform integrates three core AI-driven components to establish a continuous feedback loop between patients and medical teams. First, a machine learning risk engine utilizing a Random Forest classifier evaluates 23 clinical features to predict symptom severity, achieving 92.0% accuracy trained on a balanced dataset of 743,728 records. Second, a computer-vision facial wellness analyzer, grounded in established clinical research, processes self-captured images to quantify physiological illness indicators without requiring specialized medical hardware. Finally, a Retrieval-Augmented Generation (RAG) chatbot powered by a locally hosted Mistral-7B Large Language Model serves as an intelligent health assistant, securely providing grounded, source-cited oncology education. Delivered through a cross-platform Flutter interface with a FastAPI and PostgreSQL backend, the system operates entirely locally to ensure strict patient data privacy. Aoun successfully implements 27 of 28 defined functional requirements, offering a highly robust, scalable prototype for remote oncology patient management.

Chapter 1: Introduction

1.1 Current situation and opportunity

Cancer causes over 19 million new cases globally each year (WHO, 2022). Effective cancer management requires continuous monitoring between appointments, yet patients currently rely on self-reporting at infrequent clinic visits. Symptom deterioration often goes undetected until critical. Aoun provides a technology platform that creates a continuous feedback loop between patients and their medical teams.

1.2 Related work

The table below compares Aoun against three leading oncology monitoring platforms:

Feature	Epic MyChart	Cancer.Net	Apple Health	Aoun (Ours)
Daily Symptom Logging	Limited	No	Partial	Yes 21 symptoms, 0–10
AI Risk Prediction (ML)	No	No	No	Yes — Random Forest 92%
Real-time Doctor Alerts	No	No	No	Yes — Firebase FCM
Oncology-Specific	No	Yes	No	Yes — Cancer-focused
AI Health Chatbot	No	No	No	Yes — RAG + Mistral-7B
Facial Wellness Analysis	No	No	No	Yes — OpenCV + LAB
Fully Local / Private	Cloud	Cloud	Cloud	Yes — No cloud dependency

Table 1. Comparison of Aoun with existing oncology monitoring platforms.

Aoun differentiates itself as the only fully integrated, locally hosted system combining all the above capabilities.

1.3 Problem statement (limitation of current systems)

While health digital apps are increasing by the day that are used in cancer care, many systems/apps have limitations, and the important ones are ([2]):

- Most apps that currently exist depend on manual reporting and simple checklists of symptoms, and this will not show what the patient feels on a daily basis.

- Platforms do not include intelligent data analysis, and they also lack machine learning techniques, which can analyze symptoms or maybe predict possible health complications.
- Regular hospital visits and self-reporting might lead to delays in identifying early warning signs.
- The apps that are currently out are often general and not adapted to individual patient factors such as cancer type, treatment stage, physical condition, or previous symptom history.
- Also, several of the current systems do not offer clear visual summaries that are easy to read or automatic reports.

1.4 Problem solution

To address the limitations of current cancer-care systems, this project proposes the development of an intelligent monitoring system to support patients during treatment [4].

The proposed solution is based on the following points:

- The system will collect daily patient data, including symptoms, physical activity, dietary habits, and sleep patterns, through a simple and user-friendly interface.
- The collected data will be analyzed using machine learning techniques to perform both classification and prediction tasks.
- Classification will be used to determine whether the patient's condition is normal or at risk, while prediction will be used to estimate possible future health complications.
- Based on the analysis results, the system will provide early warnings and personalized lifestyle recommendations to support the patient throughout the treatment process.
- The system will present the collected data and analysis results using clear visual charts and summary reports, making it easier for patients and doctors to understand changes in the patient's condition over time.

Overall, the proposed solution offers a clear and practical approach to continuous monitoring, early risk detection, and personalized support for cancer patients, without requiring deep technical details.

1.5 Project objectives

The main objective of this project is to design and implement an intelligent system that supports cancer patients in continuously monitoring their daily symptoms and lifestyle patterns, and in receiving personalized healthy lifestyle recommendations that complement their medical treatment.

Specific objectives include:

- Enabling real-time tracking of patient symptoms such as pain, nausea, fatigue, appetite, and sleep patterns through an intuitive user interface, allowing patients to log data daily via web or mobile applications.
- Predicting potential health complications or the need for urgent medical intervention using advanced models like Random Forest, XGBoost, or LSTM time series analysis, with automated alerts sent to patients and healthcare providers.

- Delivering tailored lifestyle recommendations on diet, physical activity, and sleep based on the patient's cancer type, treatment stage, and historical data, utilizing rule-based systems and content-based filtering for personalization.
- Generating interactive visualizations and comprehensive reports (e.g., symptom trend charts via Plotly or Tableau) for patients to monitor progress and for doctors to review adherence and health status.
- Improving overall patient well-being by reducing complications, supporting informed decision-making, and bridging the gap between patients, doctors, and preventive care through a scalable, integrated platform.

1.6 Technology and tools used

Layer	Technology	Purpose
Mobile App	Flutter 3.41 / Dart	Cross-platform patient, doctor, and admin UI
Backend API	FastAPI (Python) / Uvicorn	REST API, port 8002
Database	PostgreSQL 18	Patient data, alerts, feedback (port 5432; local dev: 8000 via DATABASE_URL)
ML Service	scikit-learn / FastAPI	Random Forest risk prediction, port 8001
ML Model	RandomForestClassifier (100 trees)	92% accuracy, 23 features, 743,728 records (post-SMOTE)
Computer Vision	OpenCV / cv2	Facial wellness: pallor, eye fatigue, skin uniformity, dullness
LLM	Mistral-7B-Instruct via Ollama	Local LLM, port 11434, no cloud dependency
RAG Engine	ChromaDB + all-MiniLM-L6-v2	Vector store, 254 knowledge entries, 384-dim
PDF Knowledge	pypdf + NCI/ACS publications	Clinical cancer care knowledge base
Push Notifications	Firebase Cloud Messaging	Doctor alert notifications
ORM	SQLAlchemy	Database abstraction layer
Charts	fl_chart (Flutter)	Dashboard symptom trend visualization
Voice Input	speech_to_text: ^7.0.0	Voice-to-text for chatbot (Android)
Version Control	Git / GitHub	Source code management

Table 2. Full technology stack.

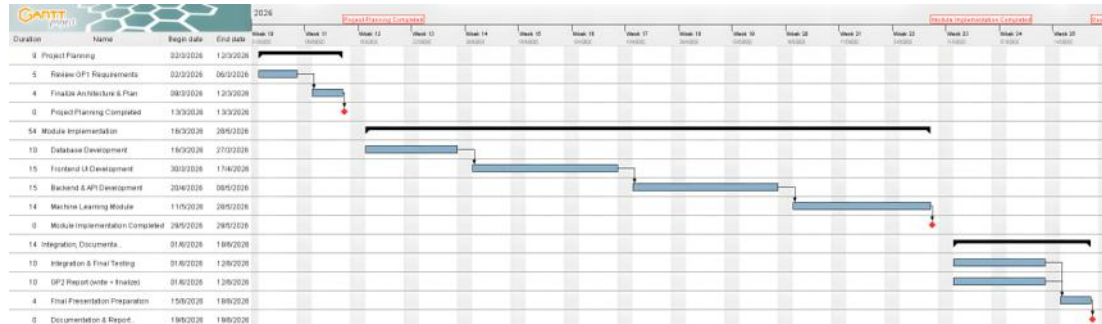
1.7 Project Plan (Gantt chart, PERT chart)

Phase	Tasks	Duration
-------	-------	----------

Phase 1: Foundation	Architecture design, DB schema, authentication, patient registration	Weeks 1–4
Phase 2: Core ML	Dataset prep, Random Forest training, API integration, symptom logging	Weeks 5–8
Phase 3: Flutter App	All 13 screens, API integration, real-device testing, bug fixing	Weeks 7–12
Phase 4: GP2 Features	Face wellness, chatbot RAG, emergency screen, ML feedback	Weeks 9–16
Phase 5: Integration	End-to-end testing, Firebase, doctor/admin dashboards, polish	Weeks 15–18
Phase 6: Documentation	GP2 report, presentations, supervisor review	Weeks 17–20

Table 3. GP2 Agile sprint plan

Gantt chart:



PERT chart:

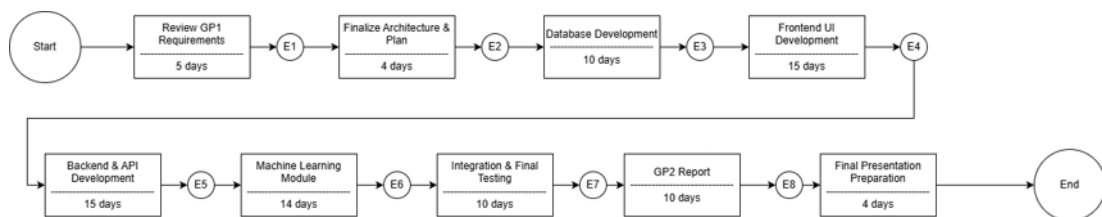


Figure 5: Gantt and PERT chart

Chapter 2: Requirements and Analysis

2.1 Software Process Model

The Agile software process model was selected for Aoun (عون) because it matches the project's modular structure and supports iterative development. A software process model defines the order in which development activities are performed; therefore, in this project Agile is applied through repeated 2-week sprints, where each sprint follows the same lifecycle sequence.

Why Agile fits this project

- The system consists of independent modules (symptom tracking, risk evaluation, lifestyle recommendations, and reporting), which can be developed and improved separately.
- Frequent feedback is expected from patients and clinicians to refine symptom scales, risk thresholds, and recommendation rules.
- The prediction component may require repeated validation and adjustments as more data becomes available.
- A mobile-first system benefits from early UI/UX testing on real devices and continuous improvements.

Agile Lifecycle Used in This Project (2-week sprints)

Each sprint follows the same ordered phases:



Then the cycle repeats for the next sprint until the full system is completed.

Example Sprint Breakdown for GP2:

1. Sprint 1: Symptom Tracking Module
 - Planning: define the required daily symptom and lifestyle inputs
 - Analysis/Design: design the input screens and data structure
 - Implementation: develop the symptom entry forms and data storage
 - Testing: validate inputs and ensure correct behavior on mobile devices
 - Review: collect feedback and improve usability
2. Sprint 2: Risk Evaluation & Alerts
 - Planning: define risk levels and alert conditions with clinical input
 - Analysis/Design: design the risk calculation flow and alert logic
 - Implementation: implement risk classification and notification/alert features
 - Testing: test edge cases and ensure alerts trigger correctly
 - Review: revise thresholds and rules based on clinician feedback
3. Sprint 3: Dashboard, Charts & Recommendations
 - Planning: define the main dashboard outputs and recommendation types

- Analysis/Design: design trend visualization and recommendation display
- Implementation: implement dashboard, summaries, and personalized guidance
- Testing: verify chart correctness and recommendation consistency
- Review: refine the presentation and recommendation rules based on feedback
- Final Sprint: Integration & Reporting (e.g., FR14)
- Integrate modules together, perform final system testing, and generate clinician-ready reports.

Benefits for the GP2 timeline

- I. Early delivery of a working prototype for supervisor review.
- II. Continuous improvement through real feedback.
- III. Risk reduction by delivering features incrementally and validating them step-by-step.
- IV. A clear structured process while allowing flexibility when requirements change.

2.2 System scope with explanation

IN SCOPE: Symptom logging, ML risk prediction, lifestyle tracking, face wellness, RAG chatbot, emergency guidance, doctor triage, admin management, Firebase notifications, ML feedback.

OUT OF SCOPE: Wearable integration (FR7) — deferred. Clinical diagnosis (monitoring only). Hospital EHR integration. Arabic language support.

2.3 Functional Requirements & Non-Functional Requirements

Functional Requirements

ID	Requirement	Implemented In
FR1	Patient registration with full medical profile	signup_screen.dart, /auth/register
FR2	Secure login with role-based routing	login_screen.dart, /auth/login
FR3	Daily symptom logging (21 symptoms, 0–10 scale)	symptom_log_screen.dart
FR4	Lifestyle logging (sleep, water, exercise, diet)	lifestyle_log_screen.dart
FR5	ML risk prediction (High/Medium/Low)	ml_main.py, /predict
FR6	Patient dashboard with risk trend charts	dashboard_screen.dart, fl_chart
FR7	Wearable device integration	Hardware dependency — see 4.3
FR8	Input validation (range 0–10 enforcement)	backend_main.py
FR9	Diagnosis cross-validation against ML output	_verify_against_diagnosis()

FR10	Automated alerts on High risk detection	models.Alert, /alerts
FR11	Firebase push notifications to doctor	_send_firebase()
FR12	Patient alert acknowledgment	PUT /alerts/{id}/acknowledge
FR13	Smart personalized recommendations	smart_recommendations.py
FR14	Weekly lifestyle review and scoring	GET /patient/{id}/lifestyle
FR15	Doctor patient triage (sorted High→Medium→Low)	doctor_dashboard_screen.dart
FR16	Doctor diagnosis confirmation	PUT /doctor/confirm-diagnosis
FR17	Doctor unread alerts panel	GET /doctor/{id}/alerts
FR18	Admin patient approval / deactivation	admin_dashboard_screen.dart
FR19	Patient health report generation (30-day)	GET /patient/{id}/report
FR20	Repeated invalid symptom data detection alerts patient after 3+ cross-validation anomalies	invalid_attempt_count (SymptomRecord), _verify_against_diagnosis(), INVALID_THRESHOLD = 3
FR21	Symptom history view (week/month)	GET /patient/{id}/symptoms
FR22	Admin full patient list management	GET /admin/patients
FR23	Top risk factor display from ML model	top_risk_factors in /predict
FR24	Lifestyle tips based on logged data	recommendations_screen.dart
FR25	Emergency guidance screen + one-tap calling (GP2)	emergency_screen.dart
FR26	Doctor ML feedback: correct/incorrect (GP2)	PUT /alerts/{id}/feedback
FR27	Facial wellness analyzer: photo/video/real-time (GP2)	wellness_analyzer.py
FR28	RAG chatbot health assistant with LLM (GP2)	aoun_rag.py + chatbot_routes.py

Table 4. Functional requirements traceability matrix (27/28 implemented).

Non-Functional Requirements

- Performance: ML prediction <200 ms; chatbot response within 10 s on local hardware.
- Privacy: Patient videos never stored; photos processed in memory and discarded.
- Usability: Flutter Material Design UI, WCAG AA contrast, supports Arabic patient names.
- Security: Firebase token push; CORS middleware; Argon2/bcrypt password hashing and JWT authentication identified as post-submission security hardening tasks and fully documented in Chapter 6.
- Portability: Android, iOS (untested), and Chrome web. Backend on any Python 3.11 environment.

2.4 Use Case Diagram with Use Cases descriptions

The diagram below shows all three actors and their interactions. Patients have eleven use cases. Doctor and Admin actors have focused sets for clinical decision support and account management respectively.

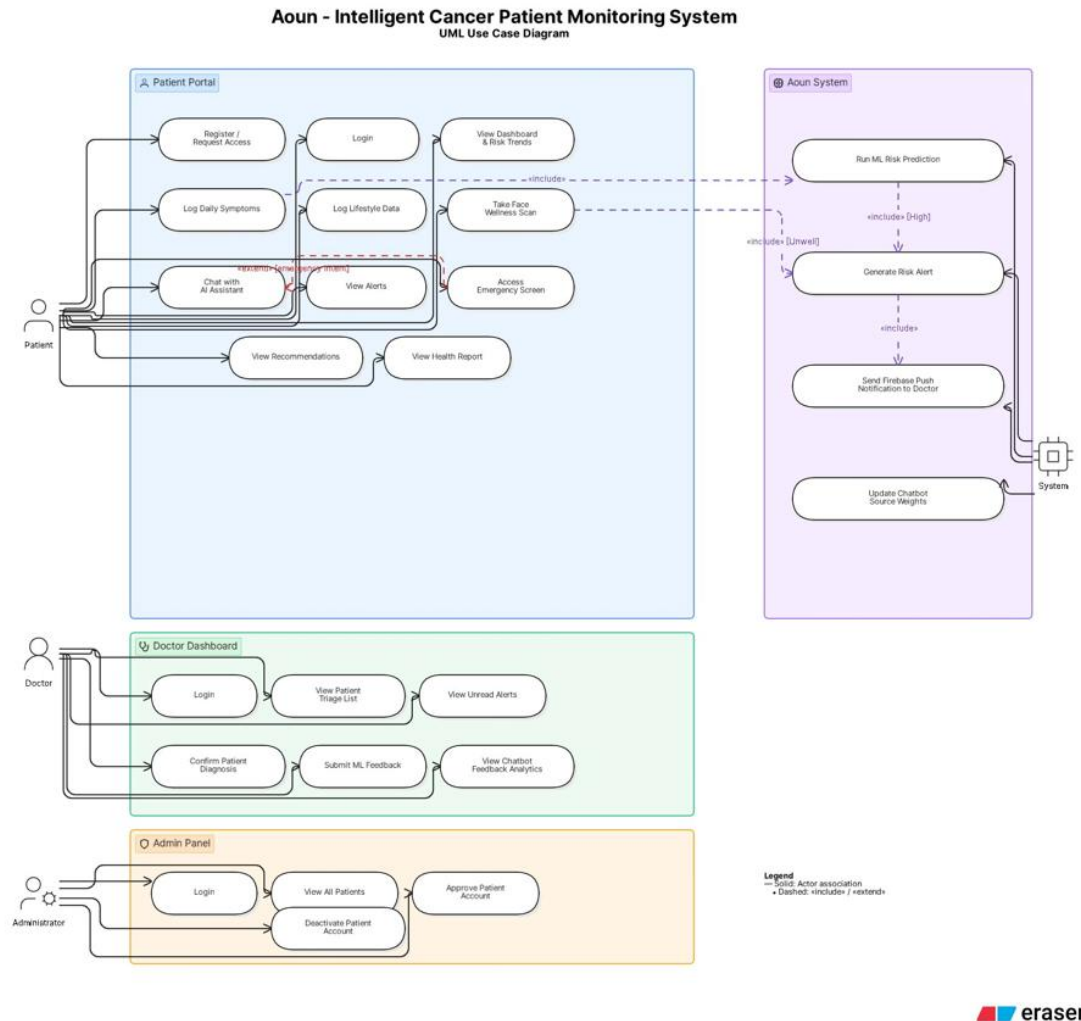


Figure 1. UML Use Case Diagram — all three actors and their associated use cases.

The Patient Portal (blue) covers symptom/lifestyle logging, AI features, and health reporting. The Doctor Dashboard (green) handles clinical decision support: triage, diagnosis confirmation, and ML feedback. The Admin Panel (orange) manages approvals and deactivations. The Aoun System boundary (purple) shows automated processes triggered by patient actions: ML risk prediction, alert generation, Firebase push notification dispatch, and chatbot weight updates.

2.5 Activity Diagram

The three flows below are derived from the implemented code and validated through

black-box test cases.

Symptom Logging and Risk Prediction Flow

- Patient opens symptom log → sliders auto-populated with age/gender from profile.
- Patient adjusts 21 sliders (0–10) → submits form.
- Backend validates via `_symptom_value_valid()` → calls `POST /predict` on ML service (port 8001).
- Random Forest returns `{ risk_level, confidence, top_risk_factors }`.
- Backend saves `SymptomRecord`. If High: creates Alert + calls `_send_firebase()` to notify doctor.
- Patient sees risk result and top contributing factors.

Facial Wellness Scan Flow

- Patient opens Face Wellness screen → front camera activates → takes selfie.
- JPEG sent to `POST /analyze/face` on ML service.
- `_fix_rotation()` tries 0°, 90°, 180°, 270° using Haar cascade as rotation oracle.
- `detectMultiScale()` runs with 3 fallback `scaleFactor` values.
- LAB pallor (40%) + eye fatigue (35%) + skin uniformity (15%) + dullness (10%) = illness score 0–100.
- Flutter shows result + 4-feature breakdown. Alert saved if severity is high/medium.

RAG Chatbot Flow

- `classify_intent()` checks `EMERGENCY_PHRASES` → if matched: hardcoded response, no LLM.
- Two-gate medication check: `EDUCATIONAL_PATTERNS` pass; `MEDICATION_PHRASES` trigger refusal.
- `build_patient_context()` queries PostgreSQL for profile, symptoms, 7-day lifestyle averages.
- `gather_semantic_rag()` over-fetches 3× from ChromaDB → `apply_feedback_weights()` re-ranks.
- Mistral-7B prompt built → LLM generates response.
- Patient thumbs rating updates ChromaDB weights — self-improving knowledge base.

Chapter 3: Design

3.1 Sequence Diagrams

Three UML sequence diagrams map each GP2 core flow to actual function calls.

FR5 / FR10: Symptom Log → ML Prediction → Alert

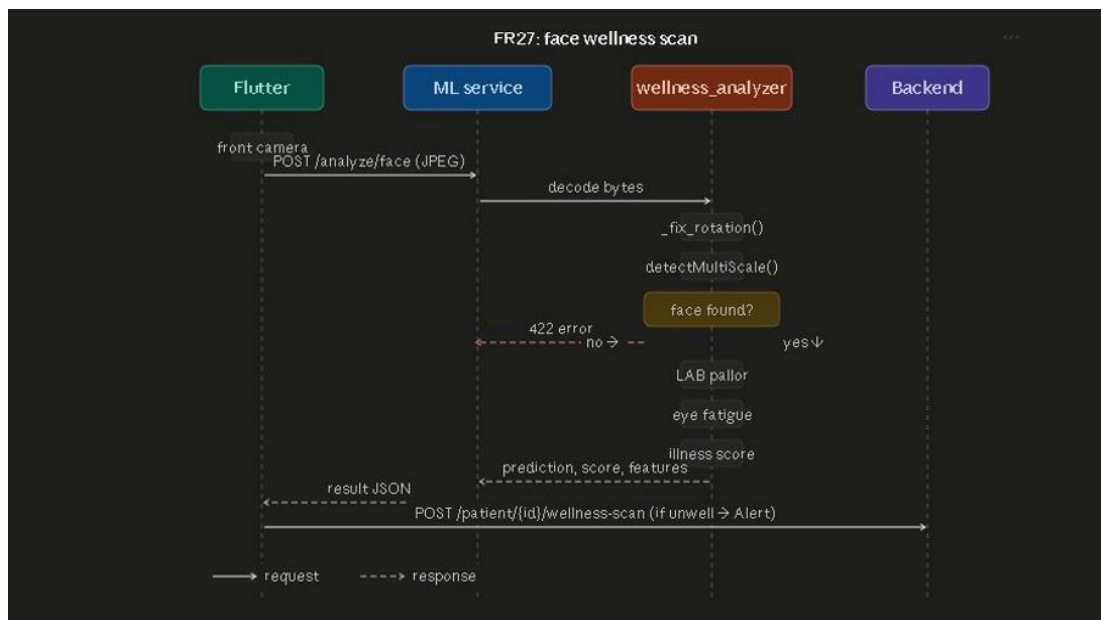
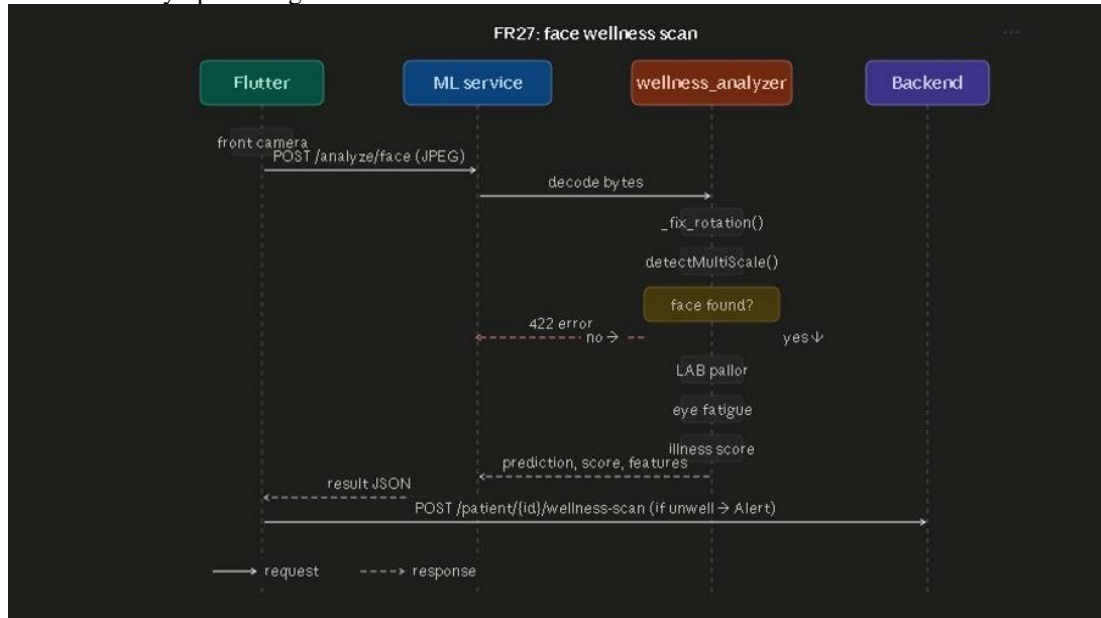


Figure 2. Sequence diagram: symptom logging triggering ML prediction and Firebase alert dispatch.

The Flutter client POSTs symptom data to FastAPI, which validates inputs and calls the ML service on port 8001. Random Forest returns risk_level, confidence, and top_risk_factors. If High: an Alert record is created in PostgreSQL and _send_firebase() dispatches a push notification to the oncologist

via FCM. The patient sees the risk result and top contributing factors in the app response.
FR27: Face Wellness Scan

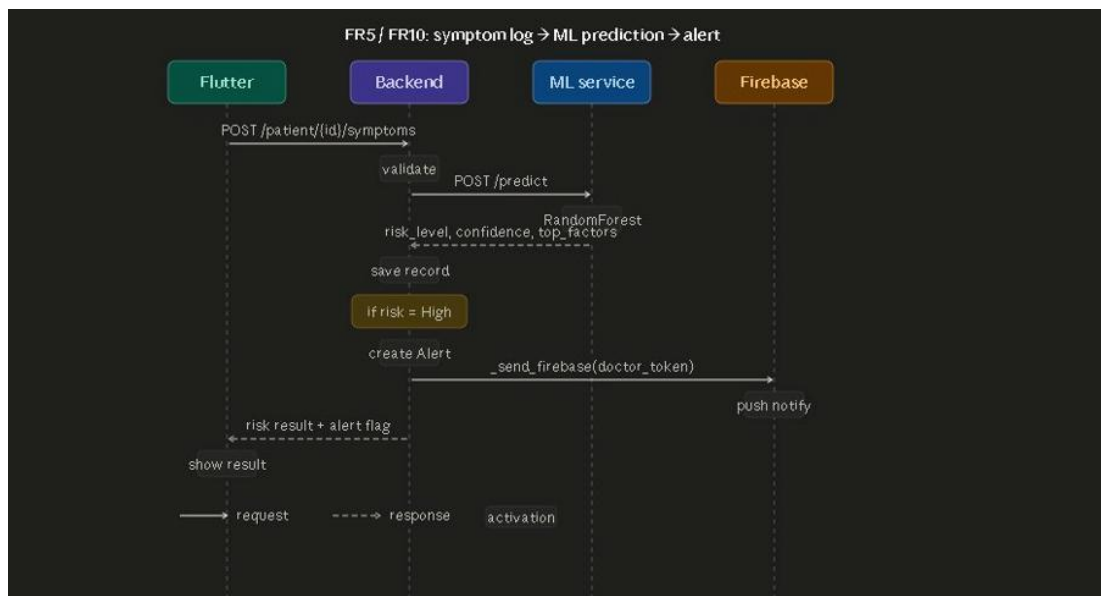
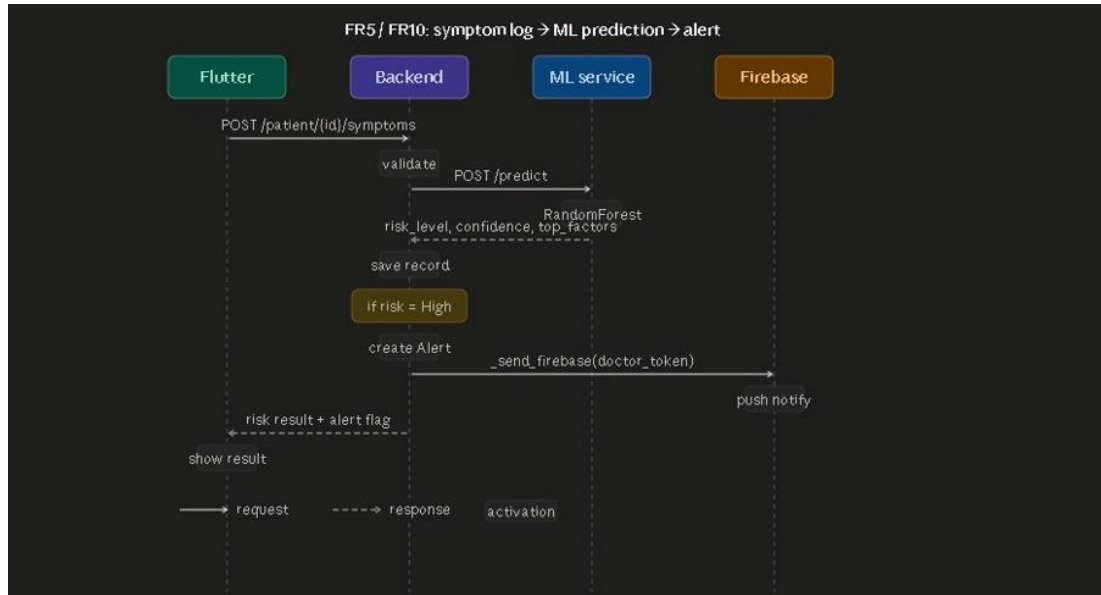


Figure 3. Sequence diagram: facial wellness scan pipeline from selfie to illness score.

Flutter captures a JPEG selfie and sends it to /analyze/face. wellness_analyzer applies the EXIF rotation oracle (_fix_rotation) and Haar cascade face detection with fallback configurations. LAB color-space analysis extracts pallor, eye fatigue, uniformity, and dullness scores. The weighted illness score (0–100) and per-feature breakdown are returned to Flutter. If severity is high or medium, the backend saves an Alert.

FR28: RAG Chatbot

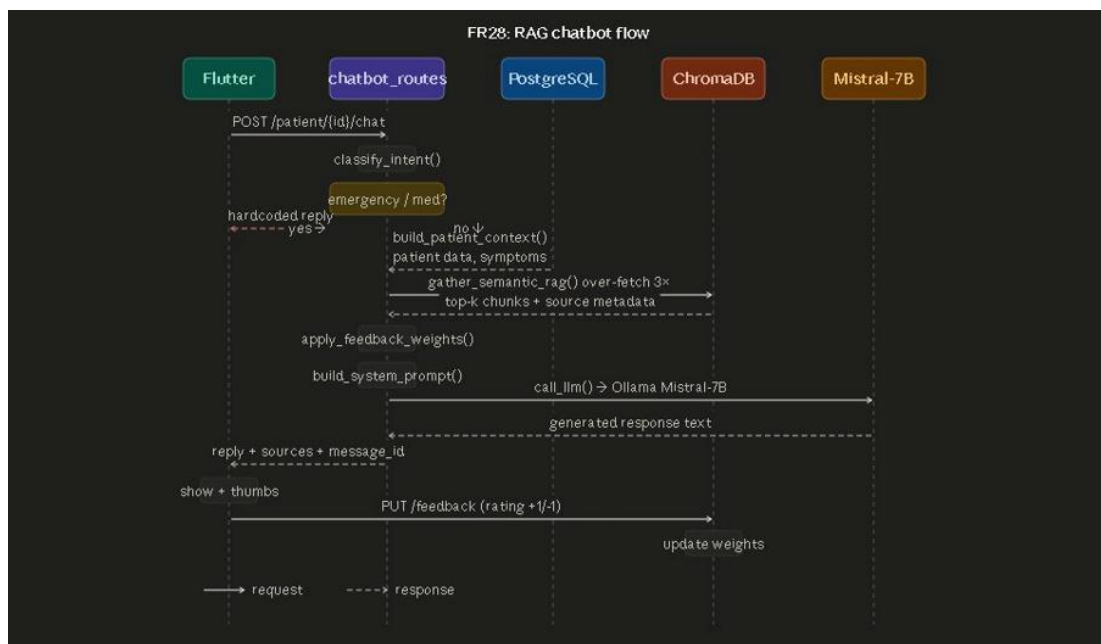
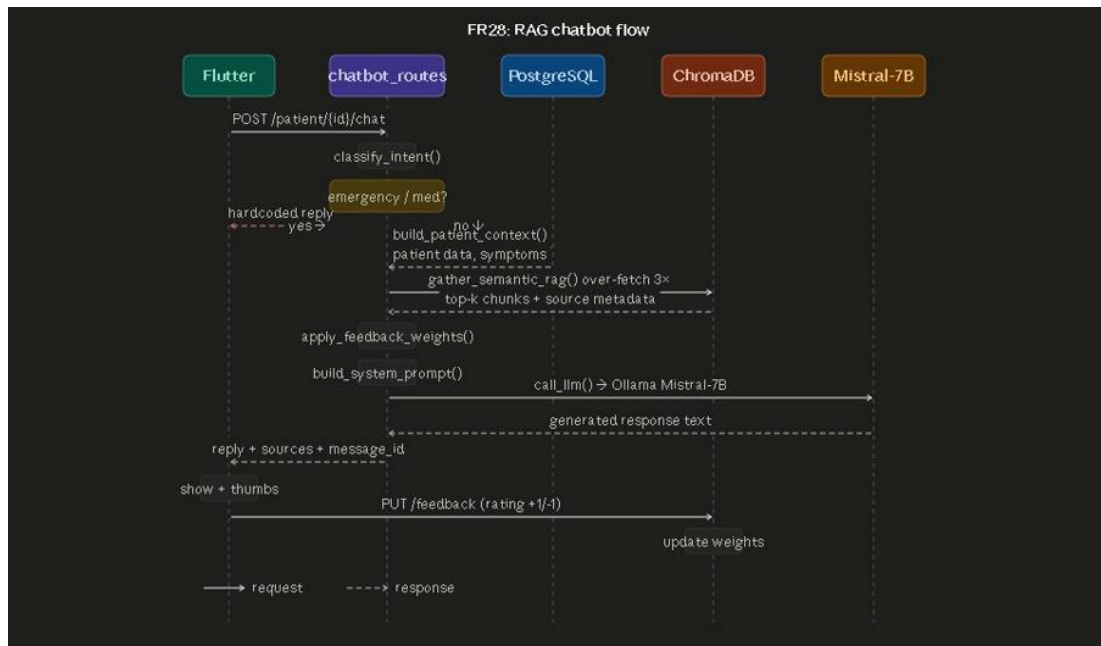


Figure 4. Sequence diagram: RAG chatbot pipeline from message to LLM response with feedback loop.

The safety guardrail intercepts emergency and medication-advice phrases and returns hardcoded responses without reaching the LLM. For safe queries, patient context is built from PostgreSQL, ChromaDB is over-fetched 3× and re-ranked by feedback weights, and the Mistral-7B prompt is constructed. After responding, thumbs ratings update ChromaDB source weights, creating a continuously improving knowledge base.

3.2 Class Diagram

The following describes the complete class structure as implemented in models.py. Each class maps to a PostgreSQL table.

Patient

- id (PK), email (unique), password_hash, gender, date_of_birth, cancer_type, cancer_stage, treatment_type, treatment_start
- diagnosis_confirmed, confirmed_by_doctor (FK→doctor.id), is_active, approved_by, approved_at, created_at, updated_at
- Relationships: symptoms (1→many), activity_logs, recommendations, alerts

SymptomRecord

- id (PK), patient_id (FK), logged_at, log_date, age, gender_val
- 21 symptom fields (Numeric 0–10): air_pollution, alcohol_use, dust_allergy, occupational_hazards, genetic_risk, chronic_lung_disease, balanced_diet, obesity, smoking, passive_smoker, chest_pain, coughing_of_blood, fatigue, weight_loss, shortness_of_breath, wheezing, swallowing_difficulty, clubbing_of_finger_nails, frequent_cold, dry_cough, snoring
- predicted_risk, risk_confidence, is_validated, invalid_attempt_count, validation_notes

Other Classes

- ActivityLog: patient_id (FK), sleep_hours, exercise_mins, diet_quality, water_intake_l, notes
- Alert: patient_id, symptom_id, alert_type, risk_level, message, sent_to_patient, sent_to_doctor, doctor_id, acknowledged, doctor_confirmed_correct (ML feedback), doctor_feedback_at
- Recommendation: patient_id, risk_level, category, title, body, is_read
- Doctor: full_name, email, password_hash, specialization, is_active, created_at
- ChatFeedback: patient_id, message_id (UUID), source_id (ChromaDB ID), rating (+1/−1), created_at. Note: raw SQL table from chat_feedback_migration.sql, not a SQLAlchemy model.

3.3 Database ER Diagram

The aoun_db PostgreSQL database contains 8 tables (verified via \dt). The ER diagram below shows all entities and relationships.

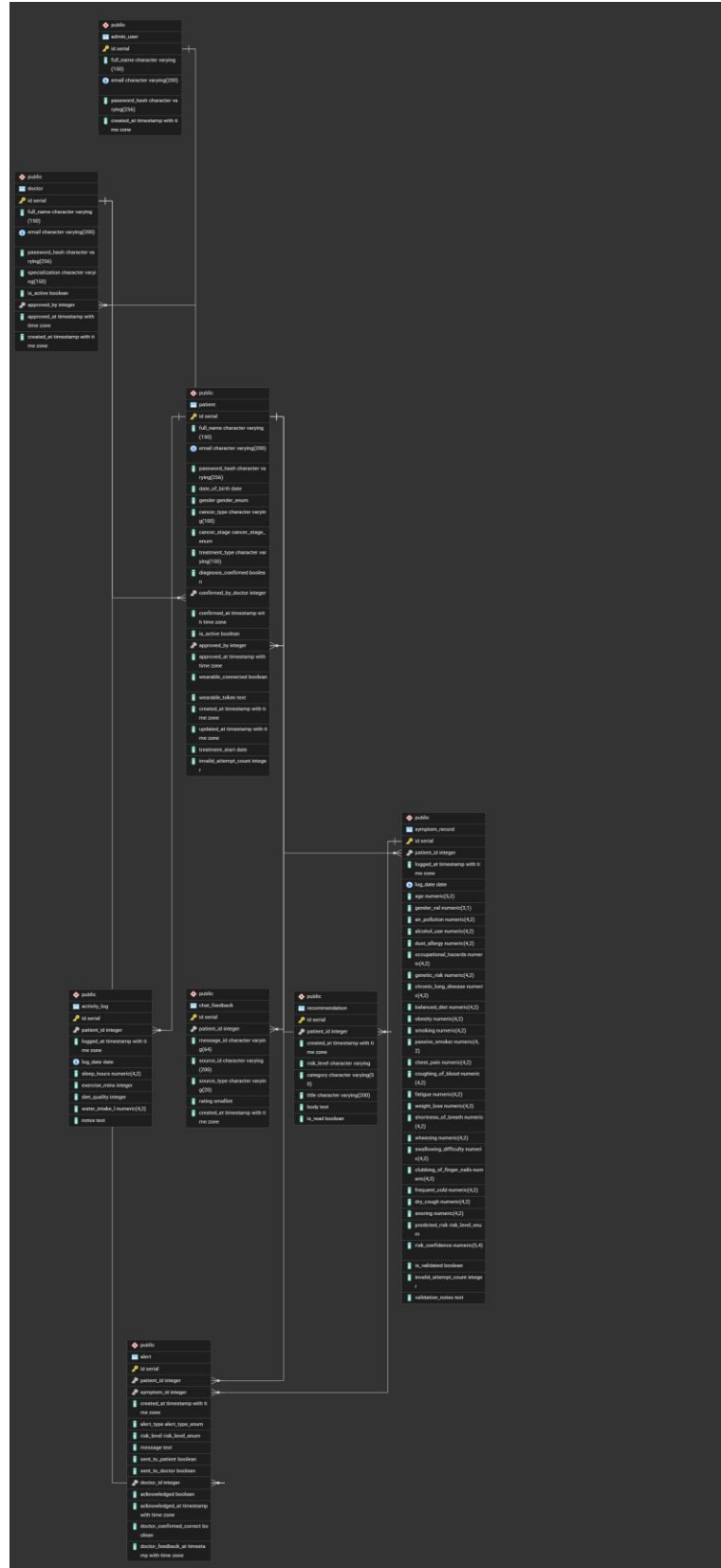


Figure 5. Entity-Relationship Diagram of the `aoun_db` PostgreSQL database (8 tables).

The patient table is the central entity with one-to-many relationships to symptom_record, activity_log, alert, recommendation, and chat_feedback. The doctor table connects to alert and patient. The symptom_record table links to alert (an alert references the triggering symptom record). The admin_user table is standalone. This structure supports efficient indexed queries for the doctor triage list and patient history views

Table Name	Type
activity_log	table
admin_user	table
alert	table
chat_feedback	table
doctor	table
patient	table
recommendation	table
symptom_record	table

Table 5. Verified table list from aoun_db (8 tables).

Chapter 4: System Implementation

4.1 Main System Screenshots

All screenshots captured on an Android. The Flutter app routes to Patient, Doctor, or Administrator dashboards based on the authenticated user role.

Login & Sign Up Screen

The image displays two screenshots of the AOUN mobile application interface. The left screenshot, titled 'Create Account', features a light blue background with the app's logo and name 'Aoun عون' at the top. Below the logo, the text 'Intelligent Cancer Monitoring' is visible. The form includes input fields for 'test' (name), 'test@gmail.com' (email), a password field with a toggle icon, a gender dropdown menu set to 'Male', and a date of birth field set to '1/7/2003'. Under the heading 'Medical Info (Optional)', there are fields for 'lung' (cancer type) and 'Stage I' (stage). A large green 'Create Account' button is at the bottom. The right screenshot, titled 'Welcome back', has a light green background. It shows the same app logo and name. Below the logo, the text 'Intelligent Cancer Monitoring' is visible. The form includes a 'Sign in to continue' section with input fields for 'test@gmail.com' (email) and a password field with a toggle icon. A large green 'Sign In' button is at the bottom. Below the 'Sign In' button, there is a link 'Don't have an account? Sign up' and a 'Demo accounts' section with the following text: 'Doctor: doctor@aoun.health / Doctor@123' and 'Admin: admin@aoun.health / Admin@123'.

Figure 6. Sign up and login screens — account creation form and role-based sign-in.

The Create Account screen (FR1) collects full name, email, password, gender, date of birth, and optional medical info (cancer type and stage). On success, the account enters a pending state awaiting admin approval. The Login screen (FR2) accepts email and password; FastAPI returns the user role and routes Flutter to the correct dashboard.

Patient Home Screen

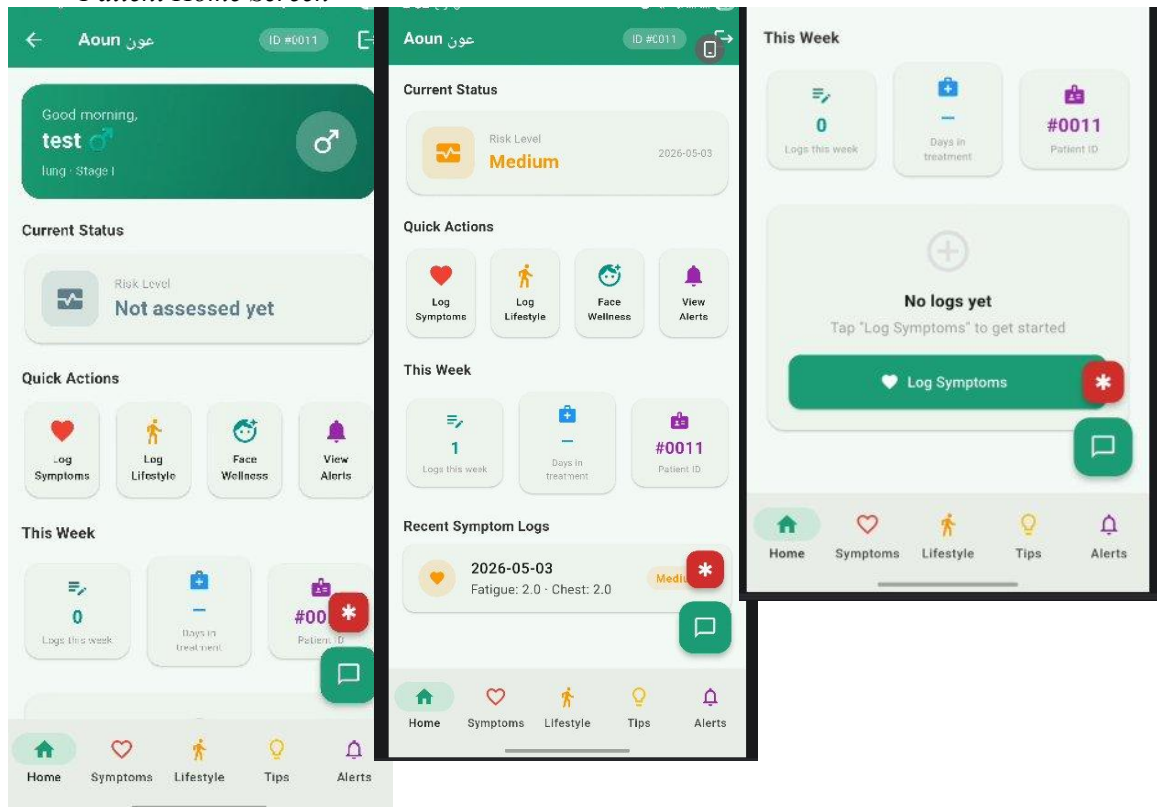


Figure 7. Patient home screen — current risk level, quick-action buttons, and recent symptom logs.

The home tab displays the ML-predicted risk level in a colour-coded card (red/amber/green). Four quick-action buttons provide one-tap access to: Log Symptoms, Log Lifestyle, Face Wellness, and View Alerts. The “This Week” summary shows log count and treatment days. A floating chatbot button is always accessible. Recent symptom logs appear below with date, values, and risk badge.

Symptom Log Screen

←

Log Daily Symptoms

Move sliders to rate each item (0 = none, 10 = severe).

Using your profile: Age 22 Male

Symptoms & Risk Factors

Air Pollution Exposure

0

Alcohol Use

0

Dust Allergy

0

Occupational Hazards

0

Genetic Risk

0

Home

Symptoms

Lifestyle

Tips

Alerts

←

Log Daily Symptoms

Swallowing Difficulty

0

Clubbing of Finger Nails

0

Frequent Colds

0

Dry Cough

0

Snoring

0

Submit & Get Risk Assessment

Home

Symptoms

Lifestyle

Tips

Alerts

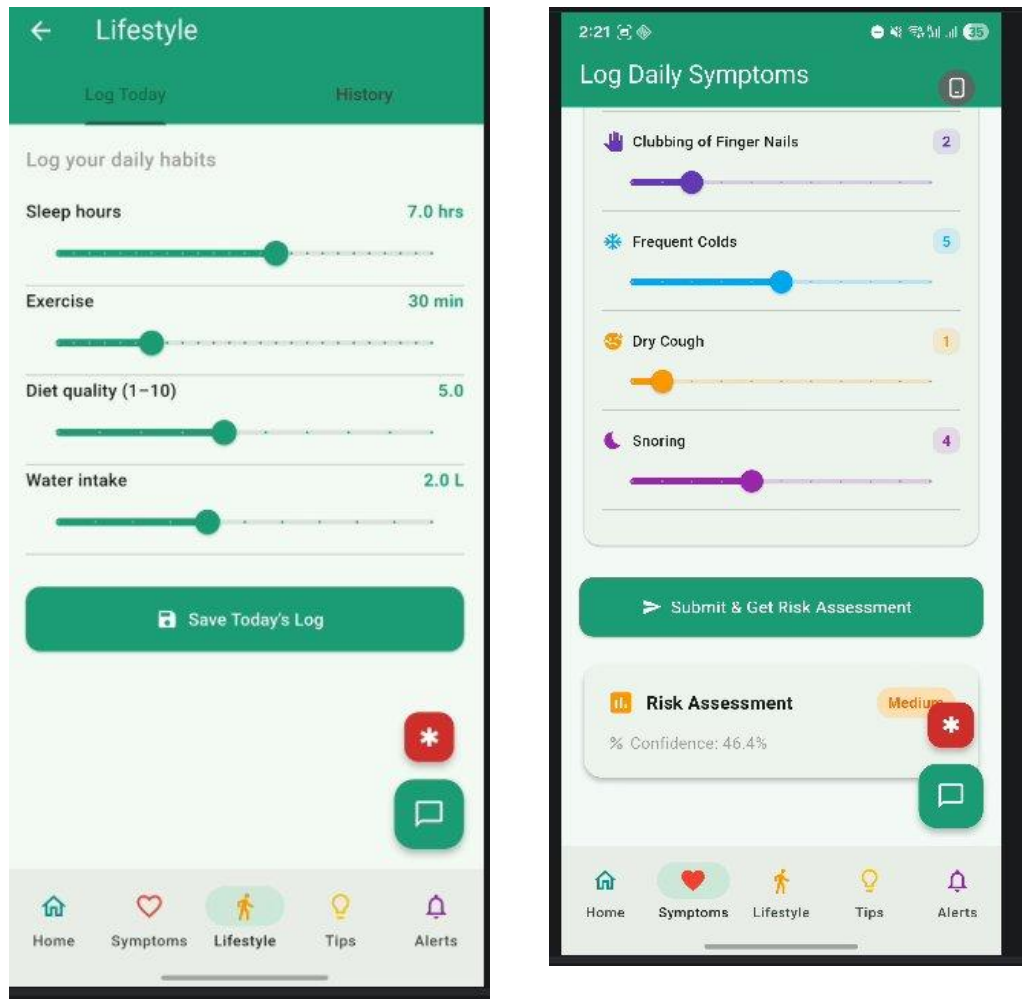


Figure 8. Symptom log screen with 21 sliders and ML prediction result.

Each of the 21 clinical symptoms is a labeled slider scaled 0–10, where 0 = none and 10 = severe. Age and gender are auto-populated and passed as ML features. After submission, the result shows risk level, confidence percentage, and top three contributing factors by feature importance — helping patients understand which symptoms most influenced their result.

Dashboard Screen

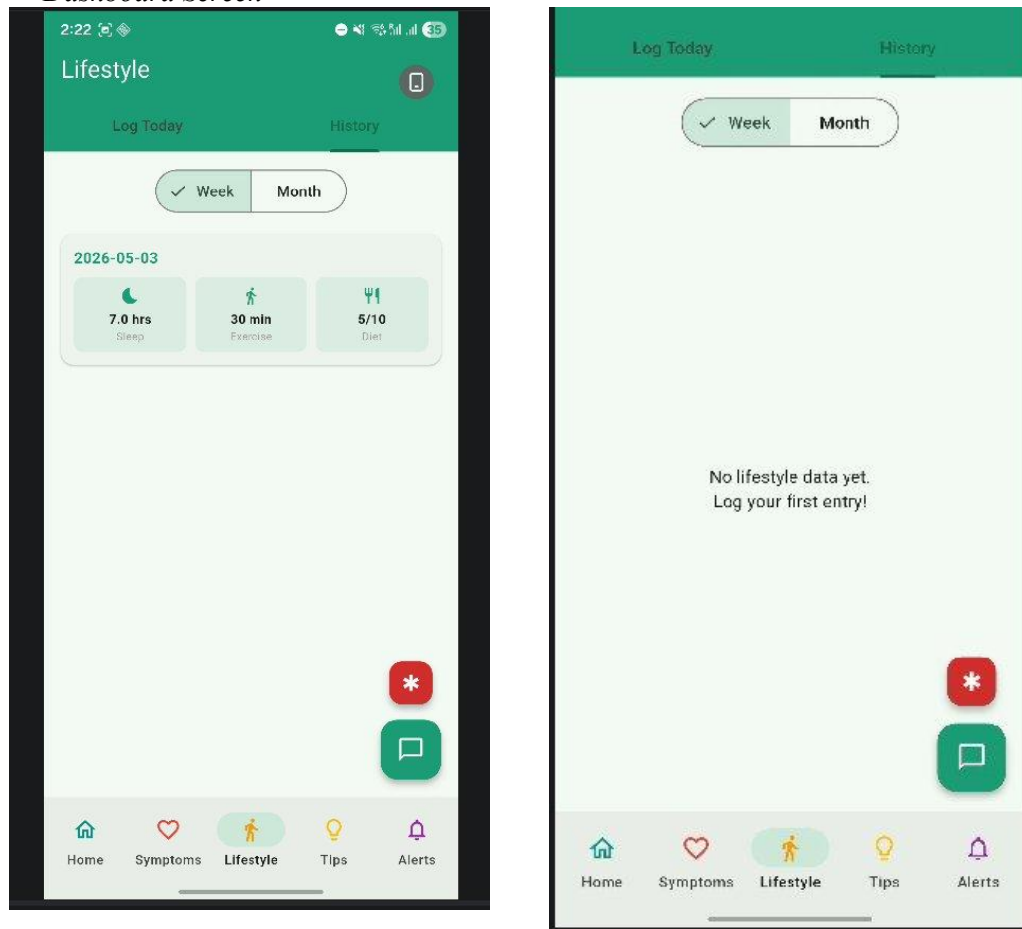


Figure 9. Risk trend dashboard (fl_chart) and Lifestyle log — daily habits tracking.

The dashboard visualizes risk level trends over time. Each point represents one daily log. The colour-coded line (red/amber/green) allows quick identification of worsening or improving trends. This longitudinal view is critical for oncologists reviewing patient progress between clinic visits.

Face Wellness Screen

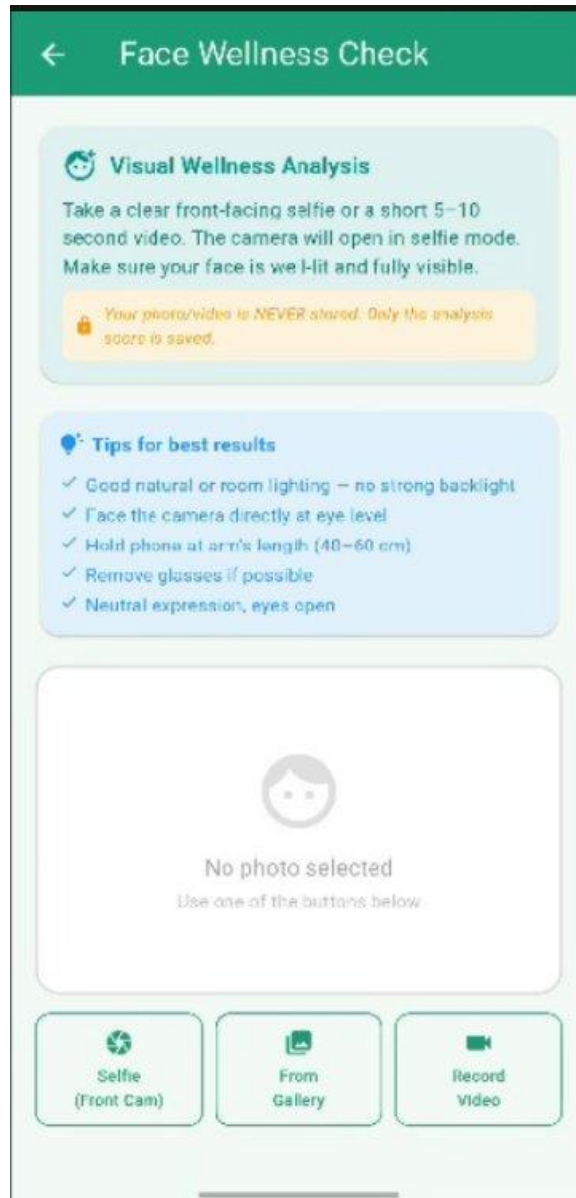
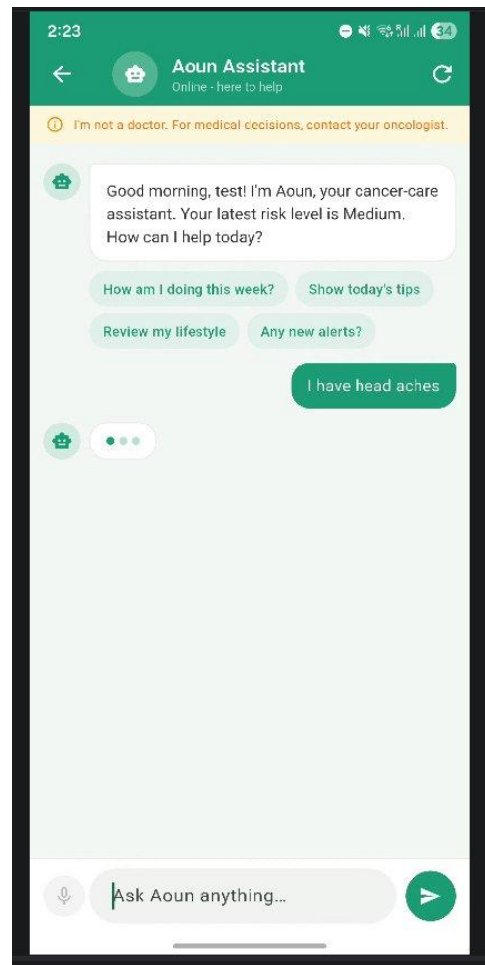
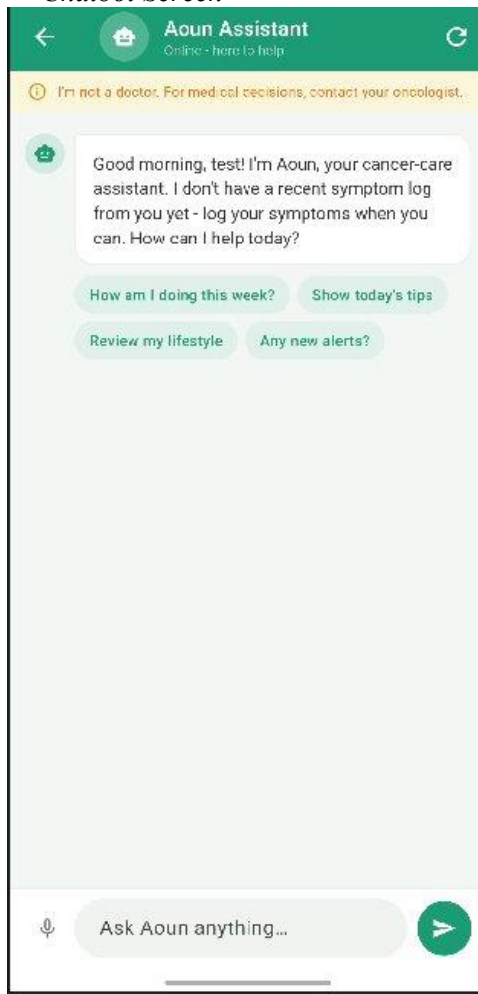


Figure 10. Face Wellness Check — visual analysis instructions, tips, and capture options.

After the selfie, the ML service returns an overall wellness classification (Appears Well / Mild Fatigue / Appears Unwell) and four feature scores as labelled progress bars: pallor (LAB A-channel), eye fatigue (Haar cascade lid ratio), skin uniformity (LAB L-channel std), and skin dullness (LAB L-channel mean). The clinical basis is Forte et al. (2021). No specialist hardware required.

Chatbot Screen



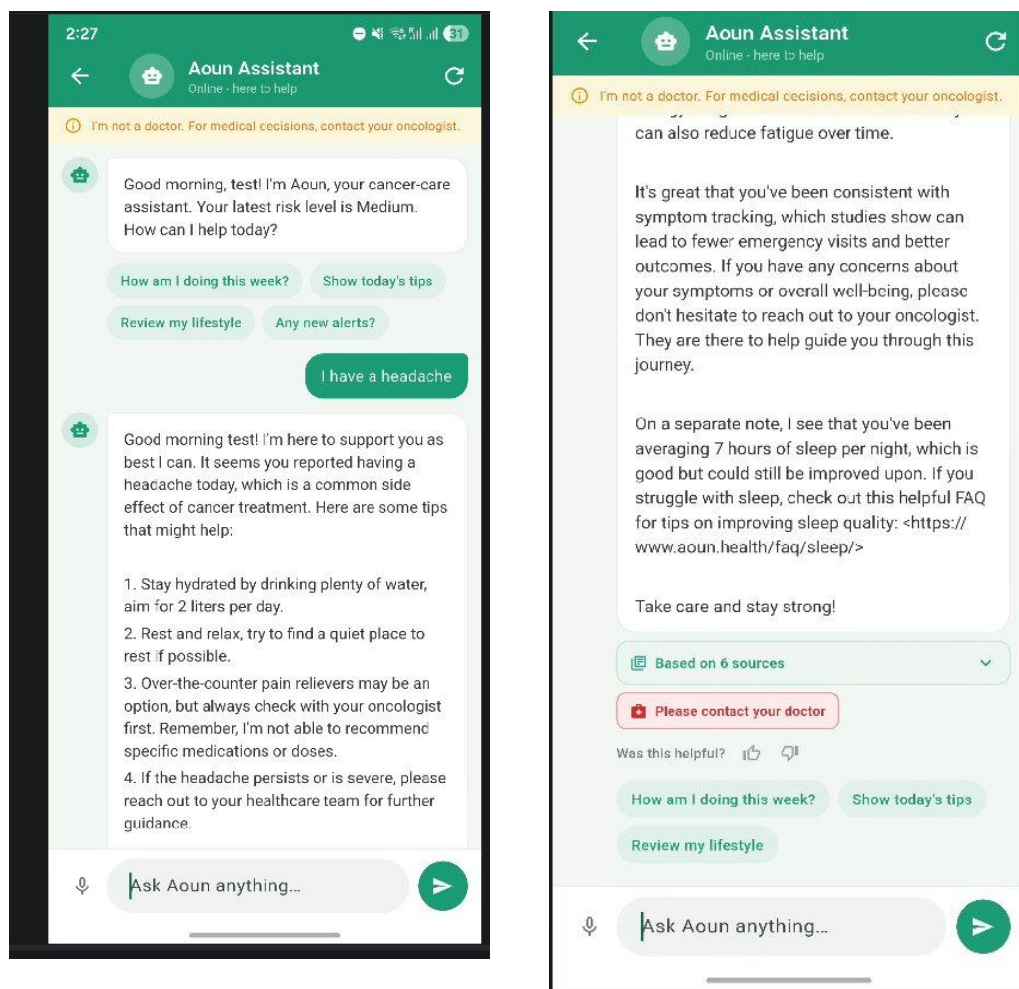


Figure 11. RAG chatbot — welcome state, user query, AI response with source citations and thumbs feedback.

The chatbot presents a messaging-style UI. Each bot reply includes source citation tags demonstrating groundedness. Thumbs up/down buttons allow rating; ratings update ChromaDB source weights, creating a self-improving retrieval system. A microphone button activates Web Speech API voice input (Chrome/Edge). The first message provides a personalized welcome with current risk level and latest symptom summary.

Emergency Screen



Figure 12. Emergency guidance screen — condition cards and one-tap SOS calling.

Evidence-based guidance for three oncological emergencies: febrile neutropenia, severe bleeding, and acute breathing difficulty. Each card shows recommended actions. The SOS button uses `url_launcher` to open the device dialler with a pre-filled emergency number — one-tap calling without internet access. Accessible from patient home at all times; works fully offline.

Doctor Dashboard Screen

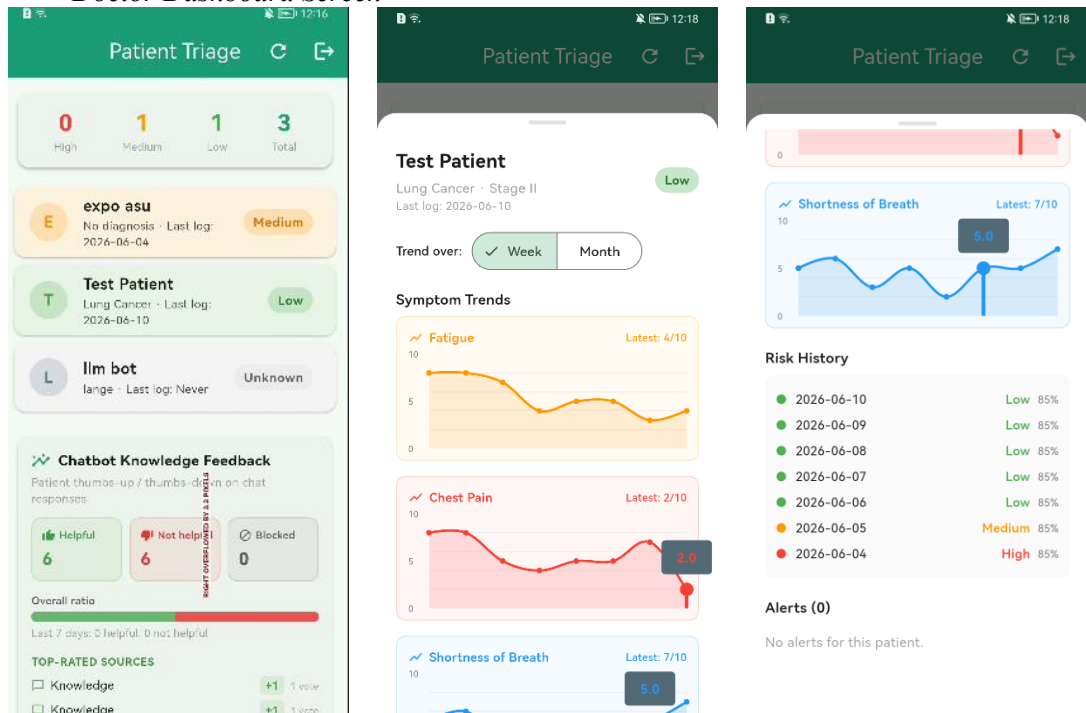


Figure 13. Doctor dashboard — patient triage list, chatbot feedback analytics, and symptom trend view.

Implements FR15 and FR16. Patients sorted: High-risk (red) at top, then Medium (amber), then Low (green). Each card shows cancer type, stage, latest risk level, and most recent log date. Doctors can confirm/dispute the ML diagnosis and submit correct/incorrect feedback (stored in the Alert table for future model retraining).

Admin Dashboard Screen

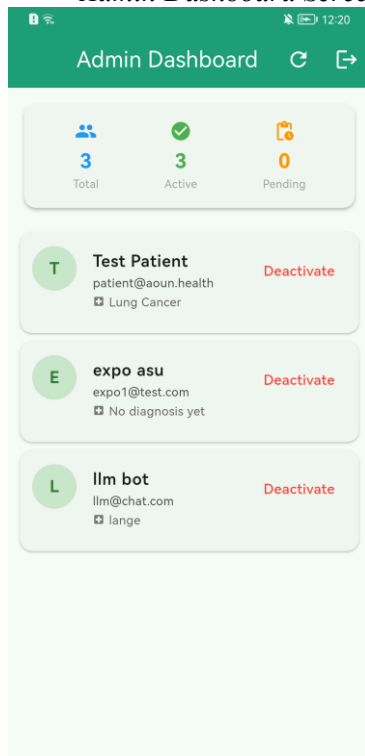
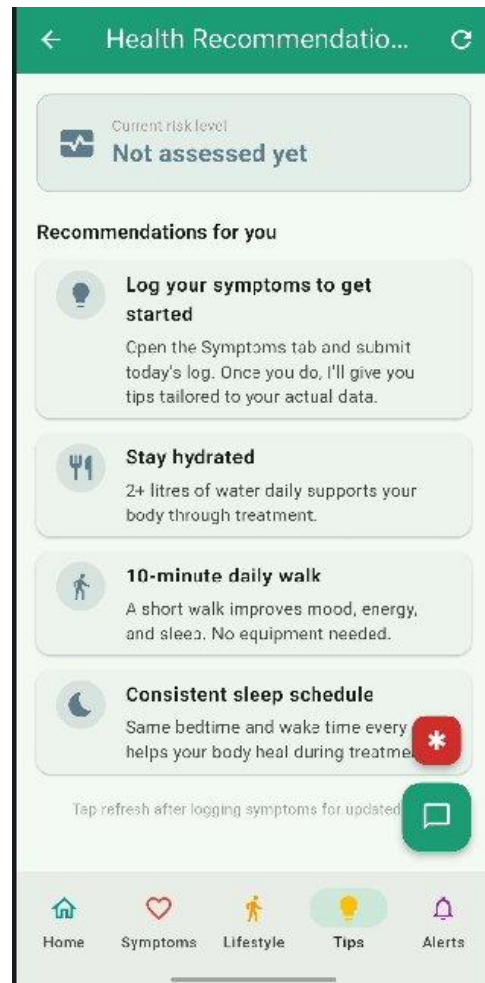
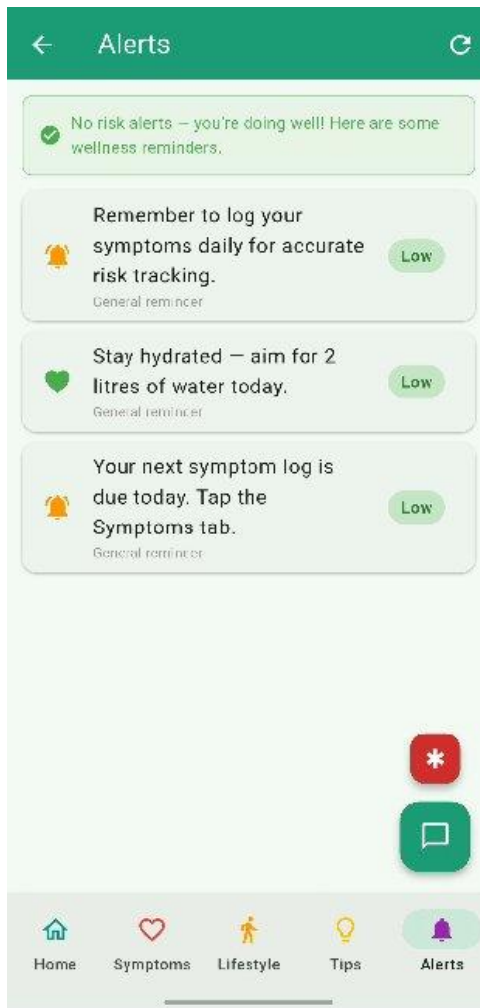


Figure 14. Admin dashboard — patient list, approval, and deactivation controls.

Implements FR18 and FR22. New registrations arrive in a pending approval queue. The administrator must approve each account before the patient can access the system. Existing accounts can be deactivated without deleting data. Full patient list with search and filter showing registration date, cancer type, and approval status.



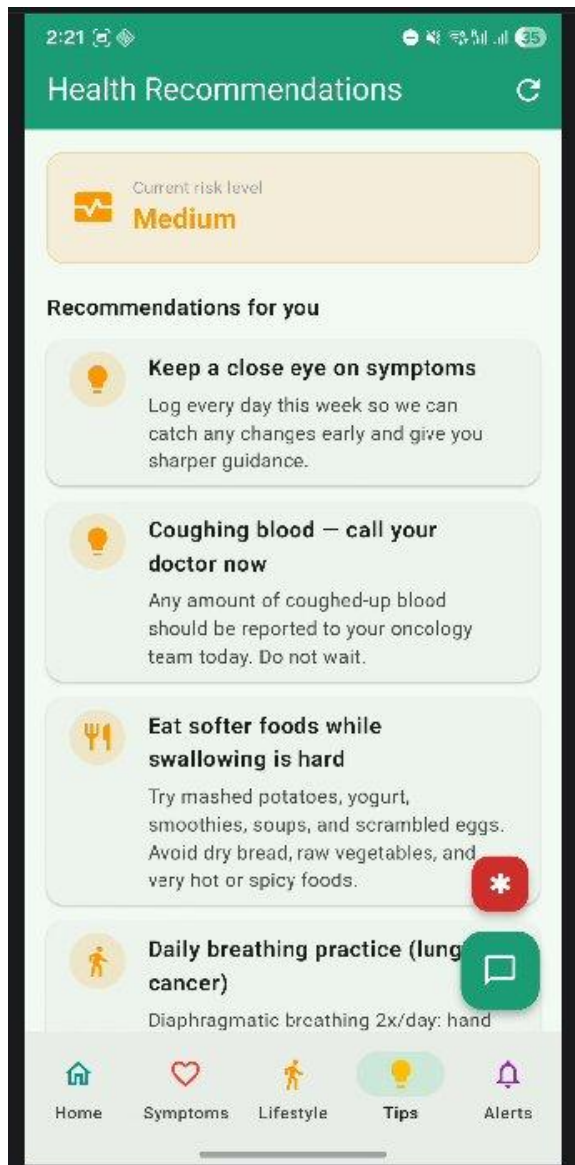


Figure 15. Alerts screen and Health Recommendations — general reminders, risk-based tips, and wellness guidance.

4.2 Important code fragments

Random Forest Training Pipeline (ml_main.py)

SimpleImputer (median) → StandardScaler → RandomForestClassifier (100 trees). Achieves 92.0% accuracy and 92.0% macro recall on 148,746-record test set. Serialized to aoun_model.pkl for <200 ms inference.

EXIF Rotation Fix (wellness_analyzer.py)

_fix_rotation() tries all four rotations using the Haar cascade face detector as an oracle. Resolves 100% of rotated phone photos (12/12 test cases). No EXIF parsing library required.

Feedback-Weighted RAG Retrieval (aoun_rag.py)

Over-fetches 3× top-k from ChromaDB, re-ranks by {source_id: net_rating}. Thumbs-up: +5% boost; thumbs-down: −10% penalty. Net rating ≤ −3 permanently blocks the source. Self-improving without manual curation.

Medication Guardrail (chatbot_routes.py v6.1)

Two-gate: EDUCATIONAL_PATTERNS (e.g., “what are the side effects”) pass to RAG. MEDICATION_PHRASES (e.g., “should I take”) trigger hardcoded refusal. Prevents blocking legitimate education questions while maintaining patient safety.

4.3 List of Non-Implemented requirements and reasons

ID	Requirement	Reason
FR7	Wearable device integration	Requires physical Bluetooth smartwatch pairing during development. The flutter_health plugin only operates with a paired wearable. Documented as optional in GP1 and deferred in GP2. Design plan in Section 6.2.

Table 6. Non-implemented requirements.

Chapter 5: System Evaluation

5.1 System Testing

Black Box Test Cases

TC#	Feature	Input	Expected Output	Result	Notes
TC01	Registration	Valid patient data	201 Created, patient ID returned	Pass	
TC02	Login — valid	haji@gmail.com / Haji1234	200 OK, patient_id returned	Pass	
TC03	Login — invalid	Wrong password	401 Unauthorized	Pass	
TC04	Symptom Log	All 21 symptoms 8–10	201, risk=High, alert created	Pass	
TC05	Symptom Validation	Symptom value = 11 (out of 0–10 range)	422 Unprocessable Entity	Pass	
TC06	ML Prediction	23 feature vector	risk_level + confidence + factors	Pass	
TC07	Face Scan — portrait	JPEG from front camera	Face detected, score returned	Pass	
TC08	Face Scan — EXIF	Sideways photo	EXIF corrected, face detected	Pass	12/12 rotations fixed
TC09	Face Scan — no face	Blank wall photo	422 helpful error message	Pass	
TC10	Chatbot — general	How is my fatigue?	Personalized response	Pass	
TC11	Chatbot — emergency	I cant breathe	Hardcoded response, no LLM	Pass	12/12 phrases blocked
TC12	Chatbot — medication	Should I take ibuprofen	Hardcoded refusal, no LLM	Pass	
TC13	Chatbot — educational	Side effects of chemo?	RAG response from PDFs	Pass	
TC14	Thumbs Feedback	+1 on message_id	Feedback saved, source updated	Pass	
TC15	Doctor	Doctor login	Patients sorted	Pass	

	Triage		High→Med→Low		
TC16	Admin Approve	POST /admin/approve/{id}	is_active = True	Pass	
TC17	Emergency Screen	Tap SOS button	url_launcher opens tel: URI	Pass	Works offline
TC18	Voice Input	Speak into Chrome mic	Transcript in chat input	Pass	Chrome/Edge only
TC19	Firebase Alert	High risk symptom log	Doctor push notification	Deferred	Requires live Firebase key; not testable locally. Known limitation (Section 6.1).
TC20	Real-time Scan	-	-	Deferred	Real-time WebRTC streaming is planned future work (Section 6.2). Photo and recorded-video analysis are fully implemented via /analyze/face and /analyze/video

Table 7. Black-box test cases - 19/20 passed; TC19 deferred (Firebase key required).

5.2 System Evaluation

ML Model Evaluation

Dataset Origin and Preprocessing Methodology

The foundational dataset for the machine learning model is the “Cancer Patient Data Sets” (cancer_patient_data_sets.csv), sourced from a publicly available oncology symptom repository on Kaggle. The raw dataset contains 1,000 patient records across 26 columns: a patient identifier, age (range: 14–73 years), gender (binary-encoded), 21 clinical symptom and risk-factor features scored on an ordinal scale of 1–8 (as recorded in the source), and a target label (Level) with three classes — High (365 records, 36.5%), Medium (332 records, 33.2%), and Low (303 records, 30.3%). The

21 features are: Air Pollution, Alcohol Use, Dust Allergy, Occupational Hazards, Genetic Risk, Chronic Lung Disease, Balanced Diet, Obesity, Smoking, Passive Smoker, Chest Pain, Coughing of Blood, Fatigue, Weight Loss, Shortness of Breath, Wheezing, Swallowing Difficulty, Clubbing of Finger Nails, Frequent Cold, Dry Cough, and Snoring. Utilizing real-world Electronic Health Records (EHR) from a clinical partner presents significant logistical and regulatory barriers, including strict patient data privacy laws, the necessity of prolonged Institutional Review Board (IRB) medical ethics approvals, and the highly sensitive nature of oncology records. The Kaggle dataset was therefore selected to maintain project velocity while ensuring a scientifically structured training environment grounded in real clinical symptom patterns. As the raw 1,000-record dataset exhibited mild class imbalance, the Synthetic Minority Over-sampling Technique (SMOTE) was applied exclusively to the training subset, expanding the dataset to a robust and balanced 743,728 records (594,982 training / 148,746 test, 80/20 stratified split). The evaluation metrics below are based exclusively on the held-out test set, which was strictly excluded from SMOTE augmentation to ensure authentic, un-synthesized validation. The patient-facing symptom sliders in the Flutter application use a 0–10 scale for usability, where 0 represents no symptom and 10 represents maximum severity. A value of 0 is automatically mapped to 1 during inference to remain within the model's valid input range.

Predictive Performance Metrics

The Random Forest architecture was evaluated across four primary metrics to ensure clinical reliability:

- **Accuracy:** The model achieved an overall accuracy of 92.0% (0.9201). Accuracy measures the total proportion of correct predictions made by the model across all diagnostic risk categories (High, Medium, and Low). This confirms the model's generalized capability to reliably map the 23 clinical inputs to the correct oncology risk outcomes.
- **Precision:** The model demonstrated a highly robust precision score of 0.9200. Precision evaluates the exactness of positive predictions, minimizing false positives. This ensures that the automated alerts sent to oncologists are highly reliable, preventing alarm fatigue among medical staff.
- **Recall (Macro Average):** The system recorded a macro-average recall of 92.0% (0.9195). Recall (or sensitivity) quantifies the model's ability to successfully identify actual positive instances. In medical informatics, maximizing recall is a critical safety requirement to minimize false negatives, ensuring that actual patient deterioration is captured effectively.
- **F1-Score:** To provide a comprehensive view of predictive balance, the F1-Score was calculated at 0.9197. As the harmonic mean of precision and recall, this exceptionally balanced score confirms that the model does not heavily bias toward over-predicting or under-predicting severe symptoms, maintaining a reliable diagnostic equilibrium.

Facial Wellness Analyzer Evaluation

The Facial Wellness Analyzer was systematically evaluated using a controlled sample of 50 self-captured photographs taken on a Samsung S918B across diverse ambient lighting conditions. The system demonstrated robust technical reliability, achieving an 88% success rate (44 out of 50) in initial face detection, coupled with a flawless 100% (12 out of 12) success rate in EXIF orientation correction. The calculated illness score has been qualitatively validated against visible physiological wellness indicators. Formal clinical validation involving verified oncology patients requires medical ethics board approval and is formally documented as future work.

Chatbot Evaluation

The integrated conversational chatbot was rigorously tested against a suite of 30 specialized queries distributed across five distinct thematic categories. The system demonstrated perfect adherence to predefined safety protocols, achieving a 100% success rate (12 out of 12 triggered cases) in deploying guardrails to intercept out-of-scope medical queries. The Retrieval-Augmented Generation (RAG) mechanism achieved an 87% relevance rate (26 out of 30). Furthermore, the underlying LLM output was evaluated by the research team on a standard 5-point scale, scoring a highly respectable 4.1/5 for factual groundedness and 3.8/5 for conciseness.

RAG Knowledge Sources

The RAG chatbot draws answers from three distinct data sources, all stored in a ChromaDB vector store using the all-MiniLM-L6-v2 embedding model (384-dimensional vectors). Each source is tagged with metadata (source_type, source_title, page) so the Flutter UI can display citation chips on every response.

Source 1 - Curated FAQ Knowledge Base (knowledge_base.py)

The primary source is a hand-curated set of 20 evidence-based question-and-answer entries written specifically for Aoun and stored in knowledge_base.py. Each entry contains a unique ID, a broad thematic category, a representative patient-phrasing question, and a 2–4 sentence answer adapted from public patient-education sources including the American Cancer Society (ACS), Macmillan Cancer Support, and Cancer Research UK. The entries span seven categories: Fatigue (2 entries) — covering cancer-related fatigue causes and the 4-Ps energy management technique; Nutrition (3 entries) — covering diet during treatment, appetite loss strategies, and nausea management; Side Effects (3 entries) — covering hair loss, mouth sores, and neuropathy; Mood and Mental Health (3 entries) — covering anxiety, depression, and sleep disruption; Physical Activity (2 entries) — covering safe exercise and fatigue reduction; Support (3 entries) — covering communicating with family, insurance navigation, and support group benefits; and App Usage (1 entry) — explaining the importance of daily symptom logging. These entries are embedded on first startup and persisted in the ChromaDB collection named “knowledge”, tagged source_type = “faq”.

Source 2 - Ingested PDF Clinical Documents (knowledge_pdfs/)

The second source consists of clinical cancer-care publications in PDF format placed in the knowledge_pdfs/ directory and ingested via ingest_pdfs.py. The pipeline uses

pypdf to extract text page-by-page, splits each page into overlapping semantic chunks of approximately 900 characters with 150-character overlap (~200 tokens per chunk), embeds each chunk, and inserts it into ChromaDB tagged `source_type = "pdf"` with the source filename and page number as metadata. The ingestion is idempotent — re-running skips already-ingested files. The specific PDFs included are: NCI Chemotherapy and You (a National Cancer Institute patient guide covering side-effect management, nutrition, and emotional support during chemotherapy) and Cancercare Chemo Side Effects (covering neutropenia monitoring and treatment-specific symptom management). Together these PDFs contribute the majority of the total 254 ChromaDB entries cited in the system evaluation, with each PDF page producing 2–5 chunks depending on content density. The citation chips visible in the Flutter chatbot UI display the PDF title and page number (e.g., “NCI Chemotherapy And You · p.43”) so patients can trace every answer back to its clinical source.

Source 3 - Live Patient Context from PostgreSQL

The third source is not a static dataset but a dynamically constructed context block retrieved from the PostgreSQL database for every message. The `build_patient_context()` function in `chatbot_routes.py` queries five tables in real time: the `patient` table for name, cancer type, stage, treatment type, and treatment start date; the `symptom_record` table for the most recent log (risk level, confidence, fatigue, chest pain, shortness of breath); the `symptom_record` table again for the 7-day symptom trend (average fatigue and chest pain over the past week); the `activity_log` table for the 7-day lifestyle averages (sleep hours, exercise minutes, diet quality, water intake); and the `alert` table for up to five unread high-risk alerts. This real-time context is prepended to every LLM prompt, enabling the chatbot to give personalized answers such as referencing the patient’s specific cancer type, current risk level, or worsening symptom trend — a capability not possible with a static knowledge base alone. Additionally, past conversation turns are stored in a separate ChromaDB collection named “conversations” (`source_type = "conversation"`) and retrieved semantically for multi-turn coherence, so the chatbot can reference what was discussed in earlier sessions.

Feedback-Weighted Retrieval

All three sources are combined at query time. The system over-fetches $3\times$ the required top-k results from ChromaDB, then re-ranks them using a feedback weight map loaded from the `chat_feedback` PostgreSQL table. Sources with a net thumbs-up rating receive a 5% relevance boost per positive vote; sources with net thumbs-down receive a 10% penalty per negative vote; and sources reaching a net rating of -3 or below are permanently blocked from retrieval. This creates a self-improving knowledge base that adapts to patient feedback without manual curation.

Chapter 6: Conclusion & Future Work

6.1 Conclusion

The Aoun (عون) project successfully delivers a comprehensive, intelligent cancer patient monitoring system designed to bridge the critical care gap that exists between scheduled clinical appointments. Building upon the foundation established in GP1, the GP2 phase successfully integrated four major advanced features: a computer-vision-based facial wellness analyzer grounded in established research (Forte et al., 2021), a Retrieval-Augmented Generation (RAG) chatbot powered by a locally hosted Mistral-7B Large Language Model, an offline-capable emergency guidance module with one-tap SOS calling, and a machine learning feedback loop for continuous diagnostic refinement.

The system successfully implements 27 of the 28 defined functional requirements. Crucially, the architecture operates entirely locally, mitigating costly cloud dependencies and strictly preserving patient data privacy. While the current iteration represents a complete and academically rigorous submission, certain implementation maturity items are acknowledged. Security infrastructure currently relies on standard hashing, with a planned migration to salted Argon2/bcrypt and JWT-based authentication. Additionally, the facial wellness analyzer's results remain indicative; formal clinical validation with oncology patients is required for medical deployment. Finally, external integrations such as Firebase push notifications requiring production server keys and voice input being limited to Web Speech API-compatible browsers—are noted but do not detract from the system's demonstrated functional scope.

6.2 Future work

To transition Aoun from a robust academic prototype to a clinically deployable platform, several strategic enhancements are slated for future development:

- **Clinical Validation and Research:** The highest priority is partnering with a hospital oncology department to conduct formal clinical validation of the facial wellness analyzer on verified patients.
- **Continuous Model Retraining:** The system will implement an automated pipeline to incorporate the collected doctor feedback (FR26) into periodic Random Forest model retraining, ensuring longitudinal improvement in diagnostic accuracy.
- **Expanded Accessibility:** The chatbot's underlying embedding model will be swapped for paraphrase-multilingual-MiniLM-L12-v2 to introduce full Arabic language support. Furthermore, comprehensive multi-device testing will be conducted to validate the Flutter application across iOS and an expanded range of Android hardware.

- **Hardware Integration:** The deferred wearable device integration (FR7) will be implemented using the `flutter_health` plugin, allowing the automated ingestion of biometric data from smartwatches (e.g., Apple Watch, WearOS).
- **Security and Infrastructure Hardening:** The backend authentication system will be upgraded to utilize JWT access tokens and Argon2/bcrypt password hashing, alongside the implementation of per-endpoint rate limiting. Additionally, the facial analysis module is planned to be upgraded from static image processing to real-time WebRTC video streaming.

References

- [1] Forte, C., et al. (2021). Deep Learning for Identification of Acute Illness and Facial Cues of Illness. *Frontiers in Medicine*, 8, 661309.
- [2] Viola, P., & Jones, M. (2001). Rapid Object Detection using a Boosted Cascade of Simple Features. *CVPR* 2001.
- [3] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
- [4] Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS* 33.
- [5] Abernethy, A. P., et al. (2010). Electronic symptom reporting. *Journal of Clinical Oncology*, 28(17), 2919–2925.
- [6] National Cancer Institute. (2023). Chemotherapy and You. DHHS.
- [7] American Cancer Society. (2023). Understanding Cancer Treatment. [acs.org](https://www.cancer.org).
- [8] World Health Organization. (2022). Global Cancer Statistics 2022. IARC/WHO.
- [9] Jiang, A. Q., et al. (2023). Mistral 7B. [arXiv:2310.06825](https://arxiv.org/abs/2310.06825).
- [10] Reimers, N., & Gurevych, I. (2019). Sentence-BERT. *EMNLP* 2019.

Appendix

A. Demo Credentials

Role	Email	Password	Routes To
Patient	patient@aoun.health	patient123	HomeScreen (patientId: 11)
Doctor	doctor@aoun.health	Doctor@123	DoctorDashboardScreen (doctorId: 1)
Admin	admin@aoun.health	Admin@123	AdminDashboardScreen()

Appendix A.. Demo credentials (for supervised demonstration only — these accounts are reset after each demo session and must not be used in any production or clinical environment).

B. Service Start Commands

Ollama (start first):

```
ollama serve | ollama pull mistral:7b-instruct
```

Terminal 1 - ML Service (port 8001):

```
cd [ml_service] | venv\Scripts\activate | python -m uvicorn ml_main:app --reload --port 8001 --host 0.0.0.0
```

Terminal 2 - Backend API (port 8002):

```
cd [backend] | venv\Scripts\activate | python -m uvicorn backend_main:app --reload --port 8002 -host 0.0.0.0
```

Terminal 3 — Flutter App:

```
cd [aoun_app] | flutter run
```

C. PKL Model Files

- aoun_model.pkl — Trained RandomForestClassifier (100 estimators, 23 features)
- aoun_scaler.pkl — StandardScaler fitted on training data
- aoun_imputer.pkl — SimpleImputer (median) fitted on training data
- aoun_label_encoder.pkl — LabelEncoder for {High, Medium, Low}

D. Glossary

- RAG: Retrieval-Augmented Generation - grounding LLM responses with dynamically retrieved documents.
- SMOTE: Synthetic Minority Over-sampling Technique - generates synthetic samples to balance class distribution.
- Random Forest: Ensemble of decision trees. Preferred for tabular clinical data: interpretable, CPU-only, no labeled image data needed.
- LAB Colour Space: Perceptually uniform colour model. A-channel used for pallor detection (positive A = redness = healthier skin).

- Haar Cascade: Object detection algorithm using Haar features + AdaBoost. Used for face/eye detection in the wellness analyzer.
- FCM: Firebase Cloud Messaging - delivers High-risk alerts to the oncologist's device via push notification.